Part A

Q1. What is an Intrusion Detection System…

1. An Intrusion Detection System (IDS) is a security tool designed to monitor and analyze data in real-time or post-event recordings to detect malicious activities, policy violations, or suspicious behavior that could indicate a security breach or threat to a system or network. The primary goal of an IDS is to detect attacks and alert the system administrators so that they can take appropriate action to mitigate the threat.
2. Yes, it is indeed possible to implement an Intrusion Detection System (IDS) using the KDD Cup 1999 dataset.
3. The workflow steps are as follows:
   a) Data Loading: Import the dataset into Apache Spark for distributed processing.
   b) Preprocessing: Convert categorical variables to numeric; Standardize continuous features for consistent scaling.
   c) Feature Selection: Use ChiSq Selector to identify and retain the most relevant features.
   d) Model Selection: Choose an SVM classifier for its effectiveness in high-dimensional spaces.
   e) Training: Split the data into training and test sets; Train the SVM model on the training set.
   f) Evaluation: Assess model performance using AUROC and AUPR metrics.
   g) Comparison: Compare SVM's performance with other classifiers like Logistic Regression.
   h) Optimization: Adjust model parameters for better accuracy and efficiency.
   i) Deployment: Deploy the trained model for real-time intrusion detection.
   j) Maintenance: Regularly update the model with new data to adapt to new threats.

Q2. Use the python urllib library to extract the KDD Cup 99 data from their web repository…

Code:

Just now (16s) 1

```
import urllib.request

urllib.request.urlretrieve("http://kdd.ics.uci.edu/databases/kddcup99/kddcup.data_10_percent.gz", "/tmp/kddcup_data.gz")
dbutils.fs.mv("file:/tmp/kddcup_data.gz", "dbfs:/kdd/kddcup_data.gz")
display(dbutils.fs.ls("dbfs:/kdd"))
```

Output:

Table +

	path	name	size	modificationTime
1	dbfs:/kdd/kddcup_data.gz	kddcup_data.gz	2144903	1720385915000

Q3. Load your data from the disk into Spark's RDD…

Code:

```
target_file = "dbfs:/kdd/kddcup_data.gz"

# Step 1: Load the data
rdd = spark.sparkContext.textFile(target_file)

# Step 2: Print 10 values from the RDD
sample_data = rdd.take(10)
for line in sample_data:
    print(line)

# Step 3: Verify the type of the RDD
print("----------------------------------------")
print(f"The data structure is of type: {type(rdd)}")
```

Output:

```
0,tcp,http,SF,181,5450,0,0,0,0,0,1,0,0,0,0,0,0,0,0,0,0,8,8,0.00,0.00,0.00,0.00,1.00,0.00,0.00,9,9,1.00,0.00,0.11,0.00,0.00,0.00,0.00,0.00,normal.
0,tcp,http,SF,239,486,0,0,0,0,0,1,0,0,0,0,0,0,0,0,0,0,8,8,0.00,0.00,0.00,0.00,1.00,0.00,0.00,19,19,1.00,0.00,0.05,0.00,0.00,0.00,0.00,0.00,normal.
0,tcp,http,SF,235,1337,0,0,0,0,0,1,0,0,0,0,0,0,0,0,0,0,8,8,0.00,0.00,0.00,0.00,1.00,0.00,0.00,29,29,1.00,0.00,0.03,0.00,0.00,0.00,0.00,0.00,normal.
0,tcp,http,SF,219,1337,0,0,0,0,0,1,0,0,0,0,0,0,0,0,0,0,6,6,0.00,0.00,0.00,0.00,1.00,0.00,0.00,39,39,1.00,0.00,0.03,0.00,0.00,0.00,0.00,0.00,normal.
0,tcp,http,SF,217,2032,0,0,0,0,0,1,0,0,0,0,0,0,0,0,0,0,6,6,0.00,0.00,0.00,0.00,1.00,0.00,0.00,49,49,1.00,0.00,0.02,0.00,0.00,0.00,0.00,0.00,normal.
0,tcp,http,SF,217,2032,0,0,0,0,0,1,0,0,0,0,0,0,0,0,0,0,6,6,0.00,0.00,0.00,0.00,1.00,0.00,0.00,59,59,1.00,0.00,0.02,0.00,0.00,0.00,0.00,0.00,normal.
0,tcp,http,SF,212,1940,0,0,0,0,0,1,0,0,0,0,0,0,0,0,0,0,1,2,0.00,0.00,0.00,0.00,1.00,0.00,1.00,1,69,1.00,0.00,1.00,0.04,0.00,0.00,0.00,0.00,normal.
0,tcp,http,SF,159,4087,0,0,0,0,0,1,0,0,0,0,0,0,0,0,0,0,5,5,0.00,0.00,0.00,0.00,1.00,0.00,0.00,11,79,1.00,0.00,0.09,0.04,0.00,0.00,0.00,0.00,normal.
0,tcp,http,SF,210,151,0,0,0,0,0,1,0,0,0,0,0,0,0,0,0,0,8,8,0.00,0.00,0.00,0.00,1.00,0.00,0.00,8,89,1.00,0.00,0.12,0.04,0.00,0.00,0.00,0.00,normal.
0,tcp,http,SF,212,786,0,0,0,1,0,1,0,0,0,0,0,0,0,0,0,0,8,8,0.00,0.00,0.00,0.00,1.00,0.00,0.00,8,99,1.00,0.00,0.12,0.05,0.00,0.00,0.00,0.00,normal.
----------------------------------------
The data structure is of type: <class 'pyspark.rdd.RDD'>
```

Q4. Split the data…

Code:

```python
split_rdd = rdd.map(lambda line: line.split(","))

# Show the total number of features (columns)
num_features = len(split_rdd.first())
print(f"Total number of features: {num_features}")

# Convert the RDD to a DataFrame
# Note: The column names are based on the KDD Cup 99 dataset documentation
columns = [
    "duration", "protocol_type", "service", "flag", "src_bytes", "dst_bytes", "land",
    "wrong_fragment", "urgent", "hot", "num_failed_logins", "logged_in",
    "num_compromised", "root_shell", "su_attempted", "num_root", "num_file_creations",
    "num_shells", "num_access_files", "num_outbound_cmds", "is_host_login",
    "is_guest_login", "count", "srv_count", "serror_rate", "srv_serror_rate",
    "rerror_rate", "srv_rerror_rate", "same_srv_rate", "diff_srv_rate",
    "srv_diff_host_rate", "dst_host_count", "dst_host_srv_count", "dst_host_same_srv_rate",
    "dst_host_diff_srv_rate", "dst_host_same_src_port_rate", "dst_host_srv_diff_host_rate",
    "dst_host_serror_rate", "dst_host_srv_serror_rate", "dst_host_rerror_rate",
    "dst_host_srv_rerror_rate", "label"
]
df = split_rdd.toDF(columns)

df.show(5)
```

Output:

```
Total number of features: 42
```

```
|       0|        tcp|  http|  SF|   181|  5450|   0|       0|   0|   0|       0|   1|       0|   0|       0|   0|       0|   0|       0|
0|       0|       0|   8|   8|    0.00|    0.00|    0.00|    0.00|    1.00|    0.00|    0.00|       9|       9|    1.00|
0.00|       0.11|    0.00|    0.00|    0.00|    0.00|    0.00|normal.|
|       0|        tcp|  http|  SF|   239|   486|   0|       0|   0|   0|       0|   1|       0|   0|       0|   0|       0|   0|       0|
0|       0|       0|   8|   8|    0.00|    0.00|    0.00|    0.00|    1.00|    0.00|    0.00|      19|      19|    1.00|
0.00|       0.05|    0.00|    0.00|    0.00|    0.00|    0.00|normal.|
|       0|        tcp|  http|  SF|   235|  1337|   0|       0|   0|   0|       0|   1|       0|   0|       0|   0|       0|   0|       0|
0|       0|       0|   8|   8|    0.00|    0.00|    0.00|    0.00|    1.00|    0.00|    0.00|      29|      29|    1.00|
0.00|       0.03|    0.00|    0.00|    0.00|    0.00|    0.00|normal.|
|       0|        tcp|  http|  SF|   219|  1337|   0|       0|   0|   0|       0|   1|       0|   0|       0|   0|       0|   0|       0|
0|       0|       0|   6|   6|    0.00|    0.00|    0.00|    0.00|    1.00|    0.00|    0.00|      39|      39|    1.00|
0.00|       0.03|    0.00|    0.00|    0.00|    0.00|    0.00|normal.|
|       0|        tcp|  http|  SF|   217|  2032|   0|       0|   0|   0|       0|   1|       0|   0|       0|   0|       0|   0|       0|
0|       0|       0|   6|   6|    0.00|    0.00|    0.00|    0.00|    1.00|    0.00|    0.00|      49|      49|    1.00|
0.00|       0.02|    0.00|    0.00|    0.00|    0.00|    0.00|normal.|
+--------+------------+-------+----+--------+--------+----+-------+----+----+----------+----+----------+----+----------
```

Q6. Get the total number of connections based on the protocol_type…

Code:

```
protocol_count_df = new_df.groupBy("protocol_type").count().orderBy("count")

# Show results in ascending order
print("Total number of connections based on protocol_type (ascending order):")
protocol_count_df.show()
```

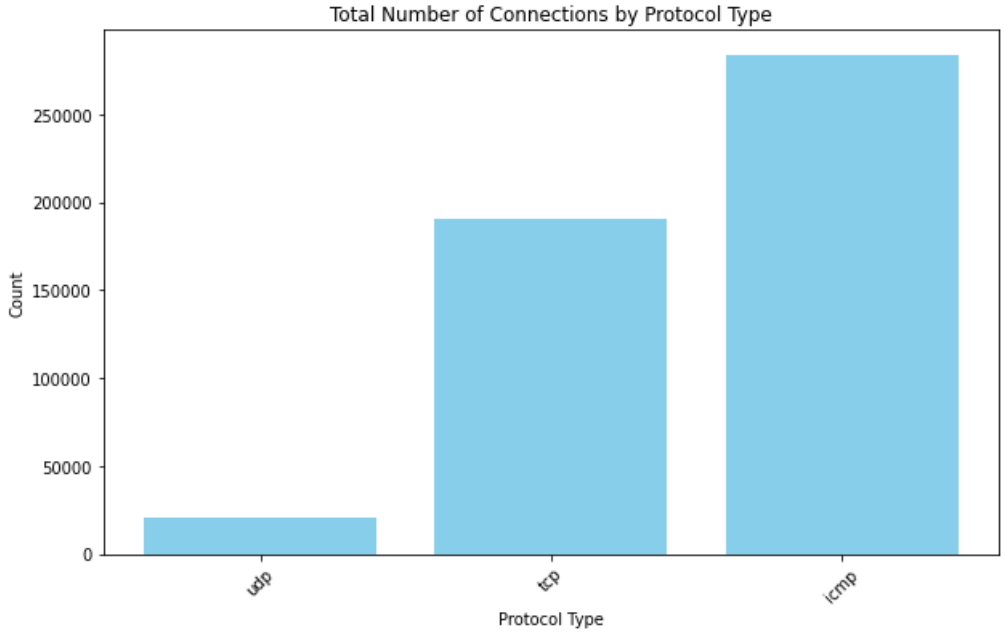
```
# Group by service and count the connections
service_count_df = new_df.groupBy("service").count().orderBy("count")

print("Total number of connections based on service (ascending order):")
service_count_df.show()
```

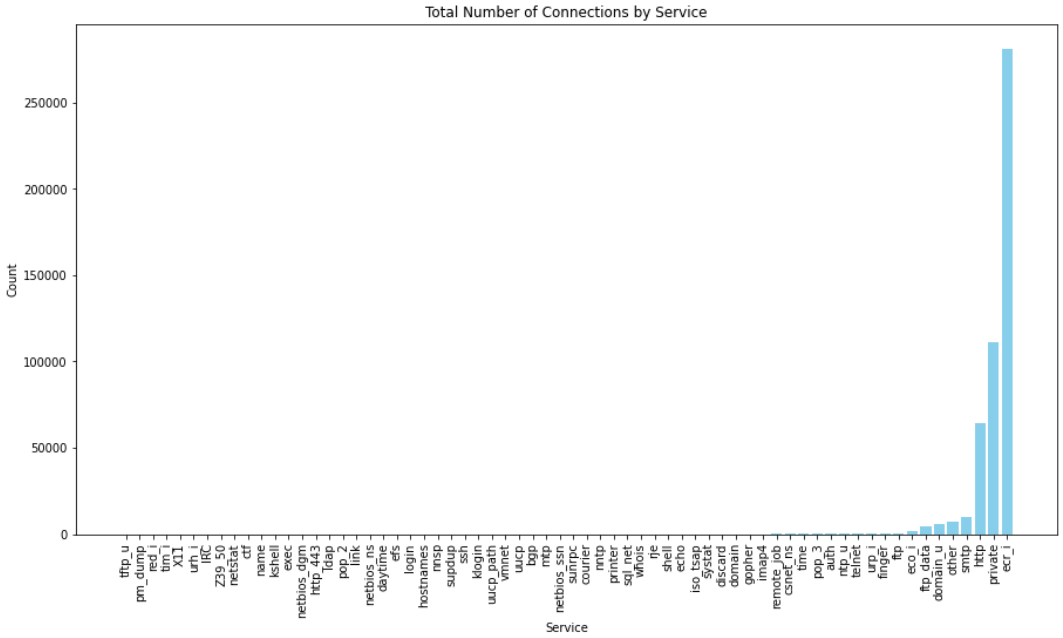
```
from matplotlib import pyplot as plt

# Convert Spark DataFrame to Pandas DataFrame for plotting
protocol_count_pd = protocol_count_df.toPandas()
service_count_pd = service_count_df.toPandas()

plt.figure(figsize=(10, 6))
plt.bar(protocol_count_pd["protocol_type"], protocol_count_pd["count"], color='skyblue')
plt.xlabel('Protocol Type')
plt.ylabel('Count')
plt.title('Total Number of Connections by Protocol Type')
plt.xticks(rotation=45)
plt.show()

# Plot the bar graph for service
plt.figure(figsize=(15, 8))
plt.bar(service_count_pd["service"], service_count_pd["count"], color='skyblue')
plt.xlabel('Service')
plt.ylabel('Count')
plt.title('Total Number of Connections by Service')
plt.xticks(rotation=90)
plt.show()
```

Output:

```
Total number of connections based on protocol_type (ascending order):
+-------------+------+
|protocol_type| count|
+-------------+------+
|          udp| 20354|
|          tcp|190065|
|         icmp|283602|
+-------------+------+
```

```
service_count_df: pyspark.s
|      red_i|    1|
|      tim_i|    7|
|        X11|   11|
|      urh_i|   14|
|        IRC|   43|
|     Z39_50|   92|
|    netstat|   95|
|        ctf|   97|
|       name|   98|
|     kshell|   98|
|       exec|   99|
|netbios_dgm|   99|
|   http_443|   99|
|       ldap|  101|
|      pop_2|  101|
|       link|  102|
| netbios_ns|  102|
|    daytime|  103|
+-----------+-----+
only showing top 20 rows
```


Total Number of Connections by Protocol Type
Count
250000
200000
150000
100000
50000
0
udp
tcp
icmp
Protocol Type


Total Number of Connections by Service
Count
250000
200000
150000
100000
50000
0
tftp_u
pm_dump
red_i
tim_i
X11
urh_i
IRC
Z39_50
netstat
ctf
name
kshell
exec
netbios_dgm
http_443
ldap
pop_2
link
netbios_ns
daytime
efs
login
hostnames
nnsp
supdup
ssh
klogin
uucp_path
vmnet
uucp
bgp
ntp
netbios_ssn
sunrpc
courier
nntp
printer
sql_net
whois
rje
shell
echo
iso_tsap
systat
discard
domain
gopher
imap4
remote_job
csnet_ns
time
pop_3
auth
ntp_u
telnet
urp_i
finger
ftp
eco_i
ftp_data
domain_u
other
smtp
http
private
ecr_i
Service

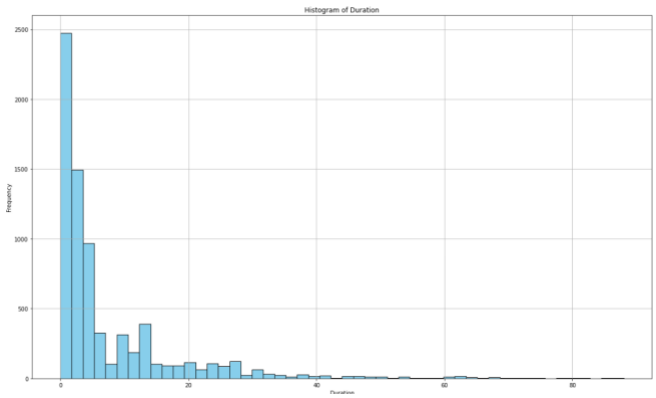
Q7. Do a further exploratory data analysis…

1. Analysis of **duration** column.

Code:

```
# Plot a histogram of the 'duration' column
import numpy as np

data_df = new_df.withColumn("duration", col("duration").cast("int"))
duration_pd = data_df.select("duration").toPandas()
non_zero_duration_pd = duration_pd[duration_pd["duration"] > 0]
max_duration = non_zero_duration_pd["duration"].quantile(0.6)


plt.figure(figsize=(20, 12))
plt.hist(non_zero_duration_pd["duration"], bins=50, range=(0, max_duration), color='skyblue', edgecolor='black')
plt.xlabel('Duration')
plt.ylabel('Frequency')
plt.title('Histogram of Duration')
plt.grid(True)
plt.show()
```

Output:



Analysis:

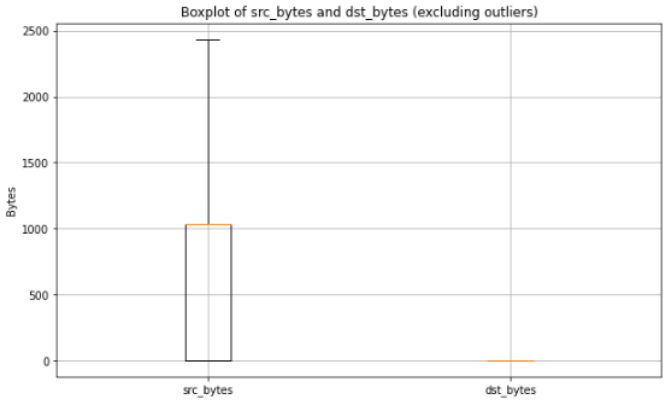
The histogram of the duration column, limited to the 60th percentile, effectively highlights the distribution of connection durations in the KDD Cup 99 dataset. The data predominantly consists of very short connections, with a rapid decline in frequency as the duration increases. The long tail and secondary peaks provide additional insights into less common but longer connections.

2. Analysis of **src_bytes** and **dst_bytes** columns.

Code:

```
# Convert columns to Pandas DataFrame for plotting
new_df = new_df.withColumn("src_bytes", col("src_bytes").cast("int"))
new_df = new_df.withColumn("dst_bytes", col("dst_bytes").cast("int"))
bytes_pd = new_df.select("src_bytes", "dst_bytes").toPandas()

Q1_src = bytes_pd["src_bytes"].quantile(0.25)
Q3_src = bytes_pd["src_bytes"].quantile(0.75)
IQR_src = Q3_src - Q1_src

Q1_dst = bytes_pd["dst_bytes"].quantile(0.25)
Q3_dst = bytes_pd["dst_bytes"].quantile(0.75)
IQR_dst = Q3_dst - Q1_dst

# Define bounds for non-outliers
lower_bound_src = Q1_src - 1.5 * IQR_src
upper_bound_src = Q3_src + 1.5 * IQR_src

lower_bound_dst = Q1_dst - 1.5 * IQR_dst
upper_bound_dst = Q3_dst + 1.5 * IQR_dst

# Filter out outliers
filtered_bytes_pd = bytes_pd[(bytes_pd["src_bytes"] >= lower_bound_src) & (bytes_pd["src_bytes"] <= upper_bound_src) &
                             (bytes_pd["dst_bytes"] >= lower_bound_dst) & (bytes_pd["dst_bytes"] <= upper_bound_dst)]


plt.figure(figsize=(10, 6))
plt.boxplot([filtered_bytes_pd["src_bytes"], filtered_bytes_pd["dst_bytes"]], labels=['src_bytes', 'dst_bytes'])
plt.ylabel('Bytes')
plt.title('Boxplot of src_bytes and dst_bytes (excluding outliers)')
plt.grid(True)
plt.show()
```

Output：



Analysis:

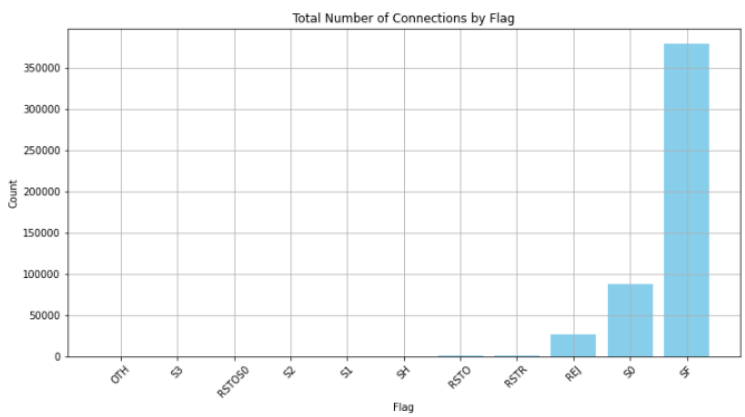
The boxplot analysis of src_bytes reveals a median value around 600 bytes, with the interquartile range (IQR) extending from approximately 200 to 1,000 bytes. The data within the whiskers extends up to about 2,500 bytes, indicating that the majority of connections fall within this range when outliers are excluded. On the other hand, dst_bytes shows most values are very close to zero, with a very small IQR, suggesting that in most connections, the amount of data sent to the destination is minimal. This highlights a significant difference in data transfer sizes between source and destination, with source bytes varying more widely while destination bytes remain consistently low.

3. Anlysis of **flag** column.

Code:

```
# Group by flag and count the occurrences
flag_count_df = new_df.groupBy("flag").count().orderBy("count")

# Convert to Pandas DataFrame for plotting
flag_count_pd = flag_count_df.toPandas()

plt.figure(figsize=(12, 6))
plt.bar(flag_count_pd["flag"], flag_count_pd["count"], color='skyblue')
plt.xlabel('Flag')
plt.ylabel('Count')
plt.title('Total Number of Connections by Flag')
plt.xticks(rotation=45)
plt.grid(True)
plt.show()
```

Output:



Analysis:
As for flat column, above figure reveals that the SF flag is overwhelmingly the most common, with over 350,000 connections, indicating successful connections with the data being fully acknowledged. The S0 flag is the second most frequent, representing connections where only the SYN packet is sent. Other flags such as REJ, RSTR, and RSTO have significantly lower counts, indicating fewer instances of rejected connections or connections reset by the originator or receiver. Flags like OTH, S1, S2, S3, SH, and RSTOS0 are rare, each with minimal occurrences. This distribution highlights that the majority of the network traffic consists of successful connections and initial SYN packets, with a smaller portion of other connection types.

Q8. Look at the label column where label == ‘normal’…

For this task, I prefer to use a Logistic Regression algorithm because it’s simple and efficient for binary classification and it also provides probabilistic outputs which are useful for understanding the confidence of predictions.

Code:

```
from pyspark.sql.functions import when, col
from pyspark.ml.feature import StringIndexer, OneHotEncoder, VectorAssembler, IndexToString
from pyspark.ml import Pipeline
from pyspark.ml.classification import LogisticRegression
from pyspark.ml.evaluation import MulticlassClassificationEvaluator

# Create a new label column
data_df = new_df.withColumn("new_label", when(new_df.label == "normal.", "normal").otherwise("attack"))
data_df = data_df.withColumn("duration", col("duration").cast("int"))
data_df = data_df.withColumn("src_bytes", col("src_bytes").cast("int"))
data_df = data_df.withColumn("dst_bytes", col("dst_bytes").cast("int"))

# Split the data into train and test sets
train_df, test_df = data_df.randomSplit([0.7, 0.3], seed=42)

# Select the columns to be used in the model
selected_columns = ["duration", "protocol_type", "service", "src_bytes", "dst_bytes", "flag", "new_label"]
train_df = train_df.select(*selected_columns)
test_df = test_df.select(*selected_columns)

# Index and encode categorical columns
indexers = [StringIndexer(inputCol=column, outputCol=column+"_index") for column in ["protocol_type", "service", "flag"]]
encoders = [OneHotEncoder(inputCol=column+"_index", outputCol=column+"_vec") for column in ["protocol_type", "service", "flag"]]

# Assemble all features into a single vector
assembler = VectorAssembler(
    inputCols=["duration", "src_bytes", "dst_bytes", "protocol_type_vec", "service_vec", "flag_vec"],
    outputCol="features"
)

# Create a logistic regression model
lr = LogisticRegression(labelCol="new_label_index", featuresCol="features", maxIter=10)

# Index the label column
label_indexer = StringIndexer(inputCol="new_label", outputCol="new_label_index")

# Convert indexed predictions back to original labels
label_converter = IndexToString(inputCol="prediction", outputCol="predicted_label", labels=["normal", "attack"])

# Create a pipeline to execute all steps
pipeline = Pipeline(stages=indexers + encoders + [assembler, label_indexer, lr, label_converter])

# Train the model
model = pipeline.fit(train_df)

predictions = model.transform(test_df)

# Evaluate the model using accuracy and F1 score
evaluator_accuracy = MulticlassClassificationEvaluator(
    labelCol="new_label_index", predictionCol="prediction", metricName="accuracy")

evaluator_f1 = MulticlassClassificationEvaluator(
    labelCol="new_label_index", predictionCol="prediction", metricName="f1")

accuracy = evaluator_accuracy.evaluate(predictions)
f1_score = evaluator_f1.evaluate(predictions)

print(f"Accuracy: {accuracy}")
print(f"F1 Score: {f1_score}")
```

Output:

```
Accuracy: 0.9893520952111385
F1 Score: 0.9894231737737884
```

Part B

Q1. Read the below statements…

1. **The answer is No**. A PaaS solution that hosts web apps in Azure provides a platform for developing, running, and managing applications without dealing with the underlying infrastructure. However, it does not inherently include professional development services to continuously add features to custom applications. Instead, it provides the tools and environment necessary for developers to build and deploy applications. Continuous addition of features to custom applications is typically handled by the development teams who use the PaaS platform.
2. **The answer is Yes**. Azure PaaS database offerings, such as Azure SQL Database and Azure Cosmos DB, are designed with built-in high availability features. These services automatically handle database maintenance, patching, and backups, and they are built on highly available and resilient infrastructure. This means they provide high availability and disaster recovery capabilities out of the box, ensuring that the database is accessible and reliable without requiring additional configuration from the user.

Q2. Read the below statement, choose the correct answer, and provide explanations.

The answer is D. Strong consistency guarantees are required.

Explanations:

a: A dynamic schema is required Relational databases use a fixed schema. For dynamic schemas, NoSQL databases are more suitable.

b: Data will be stored as key/value pairs. Key/value pair storage is typical of NoSQL databases, not relational databases.

c: Storing large images and videos. Relational databases are not optimized for storing large binary objects; object storage solutions are better.

d: Strong consistency guarantees are required. Relational databases provide strong consistency through ACID transactions, ensuring reliable and consistent data.

Q3. Read the below statement, choose the correct answer…

The answer is D. Configuring the SaaS solution.

Explanations:

a: Configuring high availability. In a SaaS model, the service provider is responsible for ensuring high availability. The infrastructure and necessary configurations to maintain high availability are managed by the SaaS provider.

b: Defining scalability rules. Scalability, including the ability to handle varying loads, is managed by the SaaS provider. Users typically benefit from the inherent scalability provided by the SaaS solution without having to define the rules themselves.

c: Installing the SaaS solution. SaaS solutions are hosted and maintained by the service provider. Users access the software via the internet and do not need to install it on their own servers or devices.

d: Configuring the SaaS solution. Users of a SaaS solution are responsible for configuring it to meet their specific needs. This includes setting up user accounts, configuring application settings, customizing workflows, and integrating with other services as needed.

Q4. Read the below statements, choose the correct answer, and provide…

1. **The answer is No**. A hybrid cloud model integrates both private and public cloud resources. A company can start with either a private or public cloud and then integrate the other. Migration from a private cloud is not a mandatory step; a company can also integrate public cloud resources with their existing on-premises infrastructure.
2. **The answer is Yes**. Public cloud services allow companies to extend their existing IT infrastructure by leveraging scalable resources such as storage, computing power, and networking capabilities provided by public cloud providers. This helps in handling increased workloads and scaling operations without the need for additional physical hardware.
3. **The answer is No**. In a public cloud model, access to resources is not limited to guest users. Authorized users within the company, including employees and administrators, can access the cloud resources. Access control mechanisms are implemented to ensure that only authorized users can access specific resources based on their roles and permissions.

Q5. Read the below statements, choose the correct answer…

a. The answer is **Fault Tolerance**. Fault tolerance refers to the capability of a system to continue operating properly in the event of a failure of some of its components. A fault-tolerant cloud service remains available and operational even when failures occur.
b. The answer is **Disaster Recovery**. Disaster recovery is the process of restoring access and functionality to IT infrastructure after a disaster or failure. It involves procedures and policies to recover from failures and ensure continuity of operations.
c. The answer is **Dynamic Scalability**. Dynamic scalability refers to the ability of a cloud service to automatically scale resources up or down based on demand. This ensures that the service performs efficiently and quickly as the demand increases.
d. The answer is **Low Latency.** Low latency refers to the minimal delay in data transfer over the network. A low-latency cloud service ensures that users can access the service quickly from the internet, providing a fast and responsive experience.